```
LineItem = Data.define(:name, :price_in_cents, :count)
li_1 = LineItem.new("Apple", 105, 3)
li_2 = LineItem.new(name: "Orange", count: 2, price_in_cents: 75)

li_1.name           # => "Apple"
li_2.price_in_cents # => 75
```

Once you have a Data instance, you can convert it to a hash with to_h.

Since the instances are immutable, you can't directly set the attributes, but you can create a new instance using with:

```
LineItem = Data.define(:name, :price_in_cents, :count)
li_1 = LineItem.new("Apple", 105, 3)
li_2 = li_1.with(name: "Orange")
```

The new instance is created using any keyword arguments passed to with and the values of the original instance for the attributes that are not included. Including a keyword argument that doesn't match an attribute raises an error.

Creating Singleton Classes

Let's look at some Ruby code:

```
class Example
end

ex = Example.new
```

When we call Example.new, we're invoking the method new on the class object Example. This is a regular method call—Ruby looks for the method new in the class of the object (and the class of Example is Class) and invokes it. So we can also invoke new directly to create a new class dynamically at runtime:

```
some_class = Class.new
puts some_class.class
```

If you pass Class.new a block, that block is used as the body of the class:

metaprogramming/class_new.rb

```
some_class = Class.new do
  def self.class_method
    puts "In class method"
  end

  def instance_method
    puts "In instance method"
  end
end

some_class.class_method
obj = some_class.new
obj.instance_method
```

produces:

```
In class method
In instance method
```